# LAB MANUAL

# ON

## FULL STACK DEVELOPMENT - 2

ACADEMIC YEAR 2026-27

## III B. Tech–I SEMESTER (R23)




**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**

**V S M COLLEGE OF ENGINEERINGAUTONOMOUS**
**RAMCHANDRAPURAM**
**E.G DISTRICT**
**533255**

| III Year I Semester | FULL STACK DEVELOPMENT - 2 | L | T | P | C |
|---|---|---|---|---|---|
| | | 0 | 1 | 2 | 2 |

**Course Objectives:**

The main objectives of the course are to

- Make use of router, template engine and authentication using sessions to develop application in Express JS.
- Build a single page application using RESTful APIs in Express JS
- Apply router and hooks in designing React JS application
- Make use of MongoDB queries to perform CRUD operations on document database

**Experiments covering the Topics:**

- Express JS – Routing, HTTP Methods, Middleware, Templating, Form Data
- Express JS – Cookies, Sessions, Authentication, Database, RESTful APIs
- React JS – Render HTML, JSX, Components – function & Class, Props and States, Styles, Respond to Events
- React JS – Conditional Rendering, Rendering Lists, React Forms, React Router, Updating the Screen
- React JS – Hooks, Sharing data between Components, Applications – To-do list and Quiz
- MongoDB – Installation, Configuration, CRUD operations, Databases, Collections and Records

**Sample Experiments:**

**1. Express JS – Routing, HTTP Methods, Middleware.**

a. Write a program to define a route, Handling Routes, Route Parameters, Query Parameters and URL building.
b. Write a program to accept data, retrieve data and delete a specified resource using http methods.
c. Write a program to show the working of middleware.

**2. Express JS – Templating, Form Data**

a. Write a program using templating engine.
b. Write a program to work with form data.

**3. Express JS – Cookies, Sessions, Authentication**

a. Write a program for session management using cookies and sessions.
b. Write a program for user authentication.

**4. Express JS – Database, RESTful APIs**

a. Write a program to connect MongoDB database using Mangoose and perform CRUD operations.
b. Write a program to develop a single page application using RESTful APIs.

**5. ReactJS – Render HTML, JSX, Components – function & Class**

a. Write a program to render HTML to a web page.
b. Write a program for writing markup with JSX.
c. Write a program for creating and nesting components (function and class).
d.

**6. ReactJS – Props and States, Styles, Respond to Events**

a. Write a program to work with props and states.
b. Write a program to add styles (CSS & Sass Styling) and display data.
c. Write a program for responding to events.

**7. ReactJS – Conditional Rendering, Rendering Lists, React Forms**

a. Write a program for conditional rendering.
b. Write a program for rendering lists.
c. Write a program for working with different form fields using react forms.

**8. ReactJS – React Router, Updating the Screen**

a. Write a program for routing to different pages using react router.
b. Write a program for updating the screen.

**9. ReactJS – Hooks, Sharing data between Components**

a. Write a program to understand the importance of using hooks.
**b.** Write a program for sharing data between components.

**10. MongoDB – Installation, Configuration, CRUD operations**

a. Install MongoDB and configure ATLAS
b. Write MongoDB queries to perform CRUD operations on document using insert(), find(), update(), remove()

**11. MongoDB – Databases, Collections and Records**

a. Write MongoDB queries to Create and drop databases and collections.
b. Write MongoDB queries to work with records using find(), limit(), sort(), createIndex(), aggregate().

**12. Augmented Programs: (Any 2 must be completed)**

a. Design a to-do list application using NodeJS and ExpressJS.
b. Design a Quiz app using ReactJS.
c. Complete the MongoDB certification from MongoDB University website.

**Text Books:**

1. Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node, Vasan Subramanian, 2nd edition, APress, O'Reilly.
2. Node.Js in Action, Mike Cantelon, Mark Harter, T.J. Holowaychuk, Nathan Rajlich,Manning Publications. (Chapters 1-11)
3. React Quickly, AzatMardan,Manning Publications(Chapters 1-8,12-14)

**Web Links:**

1. ExpressJS - https://www.tutorialspoint.com/expressjs
2. ReactJS - https://www.w3schools.com/REACT (and) https://react.dev/learn#
3. MongoDB - https://learn.mongodb.com/learning-paths/introduction-to-mongodb

# FULL STACK DEVELOPMENT – 2

## B. tech III Year I Semester

**Sample Experiments:**

**1. Express JS – Routing, HTTP Methods, Middleware.**

**a. Write a program to define a route, Handling Routes, Route Parameters, Query Parameters and URL building.**

**b. Write a program to accept data, retrieve data and delete a specified resource using http methods.**

**c. Write a program to show the working of middleware.**

## 1. Express JS – Routing, HTTP Methods, Middleware.

### a. Write a program to define a route, Handling Routes, Route Parameters, Query Parameters and URL building.

**Aim:** To understand and implement routing in Express.js using static routes, route parameters, query parameters, and URL building.

**Description:** Express.js is a lightweight and flexible Node.js framework used for developing web applications and RESTful APIs. Routing determines how an application responds to client requests for different URLs. Express provides various routing mechanisms such as static routes, dynamic route parameters, query parameters, and URL generation.

**Learning Outcomes:** After completing this experiment, students will be able to:

- Create an Express.js server.
- Define static and dynamic routes.
- Use route parameters.
- Retrieve query parameters.
- Build URLs dynamically.

**Program:**

```
const express = require('express');


const app = express();


const PORT = 3000;


// Home Route
app.get('/', (req, res) => {
```

```
    res.send('Welcome to Express Routing');
});

// Route Parameter
app.get('/student/:id', (req, res) => {
    res.send("Student ID: " + req.params.id);
});

// Query Parameter
app.get('/search', (req, res) => {
    const name = req.query.name;
    const dept = req.query.dept;

    res.send(`Student Name: ${name}, Department: ${dept}`);
});

// URL Building
app.get('/course/:branch/:year', (req, res) => {
    res.send(`Branch: ${req.params.branch}, Year: ${req.params.year}`);
});

app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
});
```

**Output:**

**URL:** http://localhost:3000/

**Output:** Welcome to Express Routing

**URL:** http://localhost:3000/student/101

**Output:** Student ID: 101

**URL:** http://localhost:3000/search?name=Ravi&dept=AIML

**Output:** Student Name: Ravi, Department: AIML

**URM:** http://localhost:3000/course/AIML/3
**Output:** Branch: AIML, Year: 3

## b. Write a program to accept data, retrieve data and delete a specified resource using http methods.

**Aim:** To develop an Express.js application that demonstrates GET, POST, and DELETE HTTP methods.

**Description:** HTTP methods are used to perform CRUD (Create, Read, Update, Delete) operations on resources. Express.js provides built-in methods to handle HTTP requests such as GET, POST, PUT, and DELETE.

**Learning Outcomes:** Students will be able to:

- Handle GET requests.
- Accept data using POST.
- Delete a resource using DELETE.
- Understand REST API basics.

**Program:**

```
const express = require('express');


const app = express();


app.use(express.json());


let students = [];


// GET Method
app.get('/students', (req, res) => {
   res.json(students);
});


// POST Method
app.post('/students', (req, res) => {
   students.push(req.body);
   res.json({
      message: "Student Added Successfully",
      data: req.body
```

```
    });
});

// DELETE Method
app.delete('/students/:id', (req, res) => {

    const id = req.params.id;

    students = students.filter(student => student.id != id);

    res.json({
        message: "Student Deleted Successfully"
    });

});

app.listen(3000, () => {
    console.log("Server Running...");
});
```

**Sample Input (POST Request):**

```
{
  "id":1,
  "name":"Nani",
  "branch":"AIML"
}
```

**Sample Output**

**GET Request**

**[]**

**POST Request**


```
{
  "message":"Student Added Successfully",
  "data":{
    "id":1,
    "name":"Nani",
    "branch":"AIML"
  }
}
```


**GET Request:**


```
[
  {
    "id":1,
    "name":"Nani",
    "branch":"AIML"
  }
]
```


**DELETE Request**:

Student Deleted Successfully

## c. Write a program to show the working of middleware.

**Aim:** To implement middleware in Express.js for logging client requests before processing them.

**Description:** Middleware functions execute between the client request and the server response. They can perform tasks such as logging, authentication, request validation, and error handling. Middleware improves application modularity and code reusability.

**Learning Outcomes:** Students will be able to:

- Understand middleware execution.
- Create custom middleware.

- Process requests using next().
- Observe middleware behavior.

**Program:**

```
const express = require('express');

const app = express();

// Custom Middleware
app.use((req, res, next) => {

    console.log("Request Received");

    next();

});

// Route
app.get('/', (req, res) => {

    res.send("Middleware Executed Successfully");

});

app.get('/about', (req, res) => {

    res.send("About Page");

});

app.listen(3000, () => {
```

```
    console.log("Server Running");

});
```

**Sample Output**

**Browser**

http://localhost:3000/

**Output:** Middleware Executed Successfully

**Browser:**

http://localhost:3000/about

**Output:** About Page

**Console Output:**

Server Running

Request Received

Request Received

## 2. Express JS – Templating, Form Data

## a. Write a program using templating engine.

## b. Write a program to work with form data.

### Part A: Express.js Templating Engine

**Aim of the Program**

To demonstrate dynamic HTML rendering in an Express.js web application using the **EJS (Embedded JavaScript)** templating engine.

**Description of the Program**

Templating engines allow developers to embed dynamic values and logic (loops, conditionals) directly into HTML files. In this application:

1. **EJS** is set up as the view engine using app.set('view engine', 'ejs').
2. A GET route (/) passes dynamic user data and an array of course modules to an EJS view template (index.ejs).
3. The template dynamically iterates over the course list and renders custom content for the user.

**Prerequisites & Folder Structure**

Create a project folder and install the required dependencies:

**BASH**

```
npm init -y
npm install express ejs
```

**Directory Layout:**

```
project-root/
├── views/
│   └── index.ejs
└── app.js
```

Program Code

app.js

**JavaScript:**

```
const express = require('express');
const path = require('path');

const app = express();
const PORT = 3000;

// Set EJS as the view engine
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

// Route: Render dynamic user dashboard
app.get('/', (req, res) => {
    const studentData = {
        name: "Alex Mercer",
        rollNo: "22A91A6601",
```

```
        branch: "CSE (AI & ML)",
        semester: "III B.Tech I-Sem",
        courses: [
            { code: "CS301", name: "Deep Learning Fundamentals" },
            { code: "CS302", name: "Express.js & Web Frameworks" },
            { code: "CS303", name: "Natural Language Processing" }
        ]
    };

    res.render('index', { student: studentData });
});

app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

**views/index.ejs**

**HTML:**

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Dashboard</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 30px; background-color: #f4f6f9; }
        .card { background: white; padding: 20px; border-radius: 8px; max-width: 500px; box-shadow: 0 2px 5px rgba(0,0,0,0.1); }
        h2 { color: #2c3e50; margin-top: 0; }
        ul { padding-left: 20px; }
        li { margin-bottom: 8px; }
```

```
    </style>
</head>
<body>
    <div class="card">
        <h2>Student Profile</h2>
        <p><strong>Name:</strong> <%= student.name %></p>
        <p><strong>Roll No:</strong> <%= student.rollNo %></p>
        <p><strong>Branch:</strong> <%= student.branch %></p>
        <p><strong>Semester:</strong> <%= student.semester %></p>

        <h3>Registered Courses</h3>
        <ul>
            <% student.courses.forEach(function(course) { %>
                <li><strong><%= course.code %>:</strong> <%= course.name %></li>
            <% }); %>
        </ul>
    </div>
</body>
</html>
```

**Output**

**Terminal Output:**

**Plaintext:**

Server running at http://localhost:3000

**Browser View (http://localhost:3000):**

**Student Profile**

**Name:** Alex Mercer

**Roll No:** 22A91A6601

**Branch:** CSE (AI & ML)

**Semester:** III B.Tech I-Sem

**Registered Courses**

- **CS301:** Deep Learning Fundamentals
- **CS302:** Express.js & Web Frameworks
- **CS303:** Natural Language Processing

**Part B: Working with Form Data in Express.js**

**Aim of the Program**

To build an Express.js application that captures user inputs via an HTML form using HTTP POST requests and processes body data using middleware (express.urlencoded).

**Description of the Program**

When submitting forms in a web application:

1. A **GET** route (/register) renders an HTML input form to the user.
2. The built-in Express middleware express.urlencoded({ extended: true }) parses incoming form submissions into req.body.
3. A **POST** route (/register) extracts the submitted parameters (studentName, email, domain) and displays a dynamic confirmation page.

**Program Code**

**app.js**

**JavaScript:**

```
const express = require('express');
const path = require('path');

const app = express();
const PORT = 3000;

// Middleware to parse URL-encoded form data
app.use(express.urlencoded({ extended: true }));

// Set EJS as view engine
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

// GET Route: Render the registration form
app.get('/register', (req, res) => {
```

```
    res.render('form');
});

// POST Route: Process form submission
app.post('/register', (req, res) => {
    const { studentName, email, domain } = req.body;

    // Pass submitted form data to the confirmation template
    res.render('result', {
        name: studentName,
        email: email,
        domain: domain,
        submittedAt: new Date().toLocaleString()
    });
});

app.listen(PORT, () => {
    console.log(`Form processing server running at http://localhost:${PORT}`);
});
```

**views/form.ejs**

**HTML**

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AI Project Registration</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 40px; background-color: #eef2f5; }
        .form-container { background: white; padding: 25px; border-radius: 8px; width: 350px; }
```

```
        .form-group { margin-bottom: 15px; }
        label { display: block; margin-bottom: 5px; font-weight: bold; }
        input, select { width: 100%; padding: 8px; box-sizing: border-box; }
        button { background-color: #28a745; color: white; border: none; padding: 10px; width: 100%; border-radius: 4px; cursor: pointer; }
        button:hover { background-color: #218838; }
    </style>
</head>
<body>
    <div class="form-container">
        <h2>Lab Project Sign-up</h2>
        <form action="/register" method="POST">
            <div class="form-group">
                <label for="studentName">Student Name:</label>
                <input type="text" id="studentName" name="studentName" required>
            </div>
            <div class="form-group">
                <label for="email">Email Address:</label>
                <input type="email" id="email" name="email" required>
            </div>
            <div class="form-group">
                <label for="domain">Specialization Domain:</label>
                <select id="domain" name="domain">
                    <option value="Computer Vision">Computer Vision</option>
                    <option value="Natural Language Processing">Natural Language Processing</option>
                    <option value="Reinforcement Learning">Reinforcement Learning</option>
                </select>
            </div>
            <button type="submit">Submit Registration</button>
        </form>
    </div>
</body>
</html>
```

**views/result.ejs**

**HTML**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Registration Confirmation</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 40px; background-color: #eef2f5; }
    .card { background: white; padding: 25px; border-radius: 8px; width: 400px; border-left: 5px solid #28a745; }
  </style>
</head>
<body>
  <div class="card">
    <h2 style="color: #28a745;">Registration Successful!</h2>
    <p><strong>Student Name:</strong> <%= name %></p>
    <p><strong>Email:</strong> <%= email %></p>
    <p><strong>Domain:</strong> <%= domain %></p>
    <p><strong>Timestamp:</strong> <%= submittedAt %></p>
    <a href="/register">Register Another Student</a>
  </div>
</body>
</html>
```

**Output**

**1. Input Form Page (http://localhost:3000/register)**

| Field | Value Entered |
|---|---|
| **Student Name** | Jane Doe |
| **Email Address** | jane.doe@example.com |
| **Specialization Domain** | Computer Vision |

**2. Processed Response Page (POST /register)**

**Registration Successful!**

**Student Name:** Jane Doe

**Email:** jane.doe@example.com

**Domain:** Computer Vision

**Timestamp:** 8/3/2026, 3:36:07 PM

[Register Another Student]

**3. Express JS – Cookies, Sessions, Authentication**

**a. Write a program for session management using cookies and sessions.**

**b. Write a program for user authentication.**

**Express.js – Cookies, Sessions, and Authentication**

**Task A: Session Management using Cookies and Sessions**

**Aim**

To implement session management in Express.js using express-session and cookie-parser to track user visits and store session state across HTTP requests.

**Description**

HTTP is a stateless protocol, meaning each request between a client and server is independent. To maintain user context (such as tracking page views or keeping a user logged in), servers use **cookies** and **sessions**.

- **Cookies:** Small pieces of data stored on the client browser.
- **Sessions:** Server-side data stores linked to a specific client via a unique Session ID stored in a signed cookie.

In this program:

1. cookie-parser parses incoming cookie headers and populates req.cookies.

2. express-session creates a session state stored on the server, sending a session ID cookie (connect.sid) to the browser.
3. The server tracks and increments the user's visit count on each page refresh.

**Program Code**

**File: package.json**

**JSON:**

```
{
  "name": "session-management-demo",
  "version": "1.0.0",
  "main": "server.js",
  "dependencies": {
    "cookie-parser": "^1.4.6",
    "express": "^4.18.2",
    "express-session": "^1.17.3"
  }
}
```

**File: server.js**

**JavaScript:**

```
const express = require('express');
const cookieParser = require('cookie-parser');
const session = require('express-session');

const app = express();
const PORT = 3000;

// Middleware setup
app.use(cookieParser());
app.use(
  session({
```

```
    secret: 'aiml_lab_secret_key_123', // Secret key to sign the session ID cookie
    resave: false,                     // Avoid resaving session if unmodified
    saveUninitialized: true,           // Save uninitialized sessions
    cookie: { maxAge: 60000 }          // Session cookie expires in 60 seconds
  })
);

// Route: Visit Tracker using Express Session
app.get('/visit', (req, res) => {
  if (req.session.views) {
    req.session.views++;
    res.send(`<h1>Welcome Back!</h1><p>You have visited this page <b>${req.session.views}</b> times.</p>`);
  } else {
    req.session.views = 1;
    res.send(`<h1>Welcome First Time User!</h1><p>Session initialized. Refresh to increment count.</p>`);
  }
});

// Route: Set a custom Cookie directly
app.get('/set-cookie', (req, res) => {
  res.cookie('user_preference', 'dark_mode', { maxAge: 900000, httpOnly: true });
  res.send('<h3>Cookie "user_preference=dark_mode" has been set!</h3>');
});

// Route: Read custom Cookie
app.get('/get-cookie', (req, res) => {
  const pref = req.cookies.user_preference || 'None';
  res.send(`<h3>Retrieved Cookie Value:</h3><p><b>user_preference:</b> ${pref}</p>`);
});

// Route: Destroy Session
app.get('/destroy-session', (req, res) => {
```

```
  req.session.destroy((err) => {
    if (err) {
      return res.send('Error clearing session');
    }
    res.send('<h3>Session destroyed successfully! Visit count reset.</h3>');
  });
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}/visit`);
});
```

**Output**

**1. Initial Visit (GET http://localhost:3000/visit)**

**Plaintext:**

Welcome First Time User!

Session initialized. Refresh to increment count.

**2. Subsequent Visits (Page Refreshed 3 times)**

**Plaintext:**

Welcome Back!

You have visited this page 4 times.

**3. Setting Cookie (GET http://localhost:3000/set-cookie)**

**Plaintext:**

Cookie "user_preference=dark_mode" has been set!

4. Getting Cookie (GET http://localhost:3000/get-cookie)

**Plaintext:**

Retrieved Cookie Value:

user_preference: dark_mode

5. Destroying Session (GET http://localhost:3000/destroy-session)

**Plaintext:**

Session destroyed successfully! Visit count reset.

**Task B: User Authentication System**

**Aim**

To write a secure Web User Authentication Application in Express.js implementing User Registration, Login, Session-based Protected Dashboard access, and Logout functionality.

**Description**

Authentication validates the identity of a user accessing a system. This program demonstrates end-to-end user authentication using Express:

1. **Registration (/register):** Collects username and password, storing them in memory (simulating a user database).
2. **Login (/login):** Validates credentials. Upon successful authentication, it attaches the user profile to req.session.user.
3. **Authorization Middleware (authMiddleware):** Intercepts requests to protected routes. If req.session.user exists, access is granted; otherwise, it responds with an HTTP 401 Unauthorized status.
4. **Logout (/logout):** Destroys the active session and clears client cookies.

**Program Code**

**File: package.json**

**JSON:**

```
{
 "name": "express-auth-demo",
 "version": "1.0.0",
 "main": "app.js",
```

```
  "dependencies": {
    "body-parser": "^1.20.2",
    "express": "^4.18.2",
    "express-session": "^1.17.3"
  }
}
```

**File: app.js**

**JavaScript:**

```
const express = require('express');
const session = require('express-session');

const app = express();
const PORT = 3000;

// In-Memory Database for demonstration
const users = [];

// Middleware
app.use(express.urlencoded({ extended: true })); // Parses HTML form submissions
app.use(express.json());                         // Parses JSON request bodies

app.use(
  session({
    secret: 'authentication_secret_key',
    resave: false,
    saveUninitialized: false,
    cookie: { maxAge: 1800000 } // Session valid for 30 minutes
  })
);
```

```
// Authentication Guard Middleware
const isAuthenticated = (req, res, next) => {
  if (req.session && req.session.user) {
    return next();
  }
  return res.status(401).send('<h2>401 Unauthorized: Please <a href="/login">login</a> to view this page.</h2>');
};

// --- ROUTES ---

// 1. Home Route
app.get('/', (req, res) => {
  res.send(`
    <h1>Express.js Auth Demo</h1>
    <a href="/register">Register</a> |
    <a href="/login">Login</a> |
    <a href="/dashboard">Dashboard (Protected)</a>
  `);
});

// 2. Registration Routes
app.get('/register', (req, res) => {
  res.send(`
    <h2>Register</h2>
    <form action="/register" method="POST">
      <input type="text" name="username" placeholder="Username" required /><br/><br/>
      <input type="password" name="password" placeholder="Password" required /><br/><br/>
      <button type="submit">Submit</button>
    </form>
  `);
});
```

```
app.post('/register', (req, res) => {
  const { username, password } = req.body;

  const existingUser = users.find(u => u.username === username);
  if (existingUser) {
    return res.send('<h3>User already exists! <a href="/register">Try again</a></h3>');
  }

  users.push({ username, password });
  res.send('<h3>Registration successful! <a href="/login">Click here to Login</a></h3>');
});

// 3. Login Routes
app.get('/login', (req, res) => {
  res.send(`
    <h2>Login</h2>
    <form action="/login" method="POST">
      <input type="text" name="username" placeholder="Username" required /><br/><br/>
      <input type="password" name="password" placeholder="Password" required /><br/><br/>
      <button type="submit">Login</button>
    </form>
  `);
});

app.post('/login', (req, res) => {
  const { username, password } = req.body;
  const user = users.find(u => u.username === username && u.password === password);

  if (user) {
    // Store user info in session
    req.session.user = { username: user.username };
    return res.redirect('/dashboard');
  }
```

```
  res.send('<h3>Invalid credentials! <a href="/login">Try again</a></h3>');
});

// 4. Protected Route (Dashboard)
app.get('/dashboard', isAuthenticated, (req, res) => {
  res.send(`
    <h2>Welcome to the Protected Dashboard, ${req.session.user.username}!</h2>
    <p>This page is only visible to logged-in users.</p>
    <a href="/logout">Logout</a>
  `);
});

// 5. Logout Route
app.get('/logout', (req, res) => {
  req.session.destroy(err => {
    if (err) {
      return res.send('Error logging out');
    }
    res.clearCookie('connect.sid');
    res.send('<h3>You have been logged out. <a href="/login">Login again</a></h3>');
  });
});

app.listen(PORT, () => {
  console.log(`Server started on http://localhost:${PORT}`);
});
```

**Output**

| Action / Route | Server Response / UI Display |
|---|---|
| **GET /dashboard** *(Without logging in)* | **401 Unauthorized: Please login to view this page.** |
| **POST /register** *(User: aiml_student, Pass: 12345)* | **Registration successful! Click here to Login** |
| **POST /login** *(With valid credentials)* | Redirects to /dashboard $\rightarrow$ **Welcome to the Protected Dashboard, aiml_student!** |
| **GET /logout** | **You have been logged out. Login again** |

**4. Express JS – Database, RESTful APIs**

**a. Write a program to connect MongoDB database using Mangoose and perform CRUD operations.**

**b. Write a program to develop a single page application using RESTful APIs.**

**Program 4(a): MongoDB Database Connection using Mongoose and CRUD Operations**

**1. Aim of the Program**

To write a Node.js program using Express.js and Mongoose to connect to a MongoDB database and perform complete CRUD (Create, Read, Update, Delete) operations on a resource.

**2. Description of the Program**

- **Mongoose** is an Object Data Modeling (ODM) library for MongoDB and Node.js. It manages relationships between data, provides schema validation, and translates between objects in code and the representation of those objects in MongoDB.
- This program sets up an Express server that connects to MongoDB via Mongoose.
- It defines a **Student** Schema and provides four REST endpoints to perform CRUD operations:
  - **POST (/students)**: Create a new student record (Create).
  - **GET (/students)**: Fetch all student records (Read).
  - **PUT (/students/:id)**: Update a student record by ID (Update).
  - **DELETE (/students/:id)**: Remove a student record by ID (Delete).

**3. Program Code**

**Prerequisites & Setup**

Before running the code, initialize a Node project and install required packages:

```
npm init -y
npm install express mongoose dotenv
```

**File: app.js**

**JavaScript**:

```
const express = require('express');
const mongoose = require('mongoose');

const app = express();
app.use(express.json()); // Middleware to parse JSON request bodies

// 1. MongoDB Connection
const MONGO_URI = 'mongodb://127.0.0.1:27017/studentDB';

mongoose.connect(MONGO_URI)
  .then(() => console.log('Successfully connected to MongoDB.'))
  .catch((err) => console.error('MongoDB connection error:', err));

// 2. Define Mongoose Schema and Model
const studentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  rollNo: { type: String, required: true, unique: true },
  branch: { type: String, required: true },
  cgpa: { type: Number, required: true }
});

const Student = mongoose.model('Student', studentSchema);

// 3. CRUD Operations / Routes
```

```
// CREATE: Add a new student
app.post('/students', async (req, res) => {
  try {
    const student = new Student(req.body);
    const savedStudent = await student.save();
    res.status(201).json({ message: 'Student created successfully', data: savedStudent });
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});

// READ: Get all students
app.get('/students', async (req, res) => {
  try {
    const students = await Student.find();
    res.status(200).json(students);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// UPDATE: Update a student by ID
app.put('/students/:id', async (req, res) => {
  try {
    const updatedStudent = await Student.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true } // Returns updated doc & runs validations
    );
    if (!updatedStudent) {
      return res.status(404).json({ message: 'Student not found' });
    }
```

```
    res.status(200).json({ message: 'Student updated successfully', data: updatedStudent });
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});

// DELETE: Remove a student by ID
app.delete('/students/:id', async (req, res) => {
  try {
    const deletedStudent = await Student.findByIdAndDelete(req.params.id);
    if (!deletedStudent) {
      return res.status(404).json({ message: 'Student not found' });
    }
    res.status(200).json({ message: 'Student deleted successfully' });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

**4. Output**

**Server Console**

**Plaintext:**

```
Server is running on http://localhost:3000
Successfully connected to MongoDB.
```

**API Testing Sample Outputs (e.g., using Postman / cURL)**

**1. POST Request (http://localhost:3000/students)**

- **Body (JSON):**

**JSON:**


```
{
  "name": "Rahul Sharma",
  "rollNo": "22A91A3501",
  "branch": "CSE (AI&ML)",
  "cgpa": 8.8
}
```


**Response (Status 201 Created):**

**JSON:**


```
{
  "message": "Student created successfully",
  "data": {
    "_id": "66a8b12f4c9a8b1234567890",
    "name": "Rahul Sharma",
    "rollNo": "22A91A3501",
    "branch": "CSE (AI&ML)",
    "cgpa": 8.8,
    "__v": 0
  }
}
```


**2. GET Request (http://localhost:3000/students)**

- **Response (Status 200 OK):**

**JSON:**


```
[
```

```
  {
    "_id": "66a8b12f4c9a8b1234567890",
    "name": "Rahul Sharma",
    "rollNo": "22A91A3501",
    "branch": "CSE (AI&ML)",
    "cgpa": 8.8,
    "__v": 0
  }
]
```

**Program 4(b): Single Page Application (SPA) using Express RESTful APIs**

**1. Aim of the Program**

To design and develop a Single Page Application (SPA) using HTML, JavaScript (Fetch API), and Express.js RESTful API endpoints.

**2. Description of the Program**

- A **Single Page Application (SPA)** dynamically rewrites the current web page with new data from the server, rather than loading entire new pages in a browser.
- In this application:
    - **Backend**: Express.js exposes RESTful API endpoints for managing a task/todo list.
    - **Frontend**: A single index.html file uses vanilla JavaScript and fetch() to consume REST APIs asynchronously without reloading the page.

**3. Program Code**

**File Structure**

**Plaintext:**

```
project-folder/
        server.js
                public/
                        index.html
```

**File 1: server.js (Express Backend API)**

**JavaScript:**

```
const express = require('express');
const path = require('path');

const app = express();
app.use(express.json());

// Serve static frontend files from 'public' folder
app.use(express.static(path.join(__dirname, 'public')));

// In-memory data store for demonstration
let tasks = [
  { id: 1, title: 'Learn Node.js REST APIs' },
  { id: 2, title: 'Build an Express SPA' }
];

// GET: Fetch all tasks
app.get('/api/tasks', (req, res) => {
  res.json(tasks);
});

// POST: Add a new task
app.post('/api/tasks', (req, res) => {
  const newTask = {
    id: Date.now(),
    title: req.body.title
  };
  tasks.push(newTask);
  res.status(201).json(newTask);
});

// DELETE: Delete a task by ID
app.delete('/api/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
```

```
  tasks = tasks.filter(t => t.id !== taskId);
  res.json({ message: 'Task deleted successfully' });
});

const PORT = 3000;
app.listen(PORT, () => console.log(`SPA Server running at http://localhost:${PORT}`));
```

**File 2: public/index.html (Single Page Frontend)**

**HTML**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Task Manager - SPA</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 40px; max-width: 500px; }
    input { padding: 8px; width: 70%; }
    button { padding: 8px 12px; cursor: pointer; }
    ul { list-style-type: none; padding: 0; }
    li { display: flex; justify-content: space-between; padding: 8px 0; border-bottom: 1px solid #ccc; }
    .del-btn { background: #ff4d4d; color: white; border: none; border-radius: 3px; }
  </style>
</head>
<body>

  <h2>Task Manager (SPA)</h2>

  <div>
    <input type="text" id="taskInput" placeholder="Enter new task..." />
    <button onclick="addTask()">Add Task</button>
```

```
</div>

<ul id="taskList"></ul>

<script>
  const API_URL = '/api/tasks';

  // 1. Fetch and Display Tasks (Read)
  async function fetchTasks() {
    const res = await fetch(API_URL);
    const tasks = await res.json();
    const list = document.getElementById('taskList');
    list.innerHTML = '';

    tasks.forEach(task => {
      const li = document.createElement('li');
      li.innerHTML = `
        <span>${task.title}</span>
        <button class="del-btn" onclick="deleteTask(${task.id})">Delete</button>
      `;
      list.appendChild(li);
    });
  }

  // 2. Add Task (Create)
  async function addTask() {
    const input = document.getElementById('taskInput');
    const title = input.value.trim();
    if (!title) return alert('Please enter a task title');

    await fetch(API_URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
```

```
      body: JSON.stringify({ title })
    });

    input.value = '';
    fetchTasks(); // Refresh list dynamically without reloading page
  }

  // 3. Delete Task (Delete)
  async function deleteTask(id) {
    await fetch(`${API_URL}/${id}`, { method: 'DELETE' });
    fetchTasks(); // Refresh list dynamically
  }

  // Initial load
  fetchTasks();
 </script>
</body>
</html>
```

**4. Output**

**Web Browser View (http://localhost:3000)**

**Plaintext:**

Task Manager (SPA)

[ Enter new task... ] [ Add Task ]

• Learn Node.js REST APIs [ Delete ]

• Build an Express SPA [ Delete ]

**SPA Interaction Behavior:**

1. When entering "Complete AI/ML Lab Assignment" and clicking **Add Task**, an asynchronous POST request is sent via JavaScript fetch(). The new task appears in the list instantly without a browser refresh.

2. Clicking **Delete** sends an asynchronous DELETE request to remove the item from the server and updates the DOM dynamically.

## 5. ReactJS – Render HTML, JSX, Components – function & Class

**a. Write a program to render HTML to a web page.**

**b. Write a program for writing markup with JSX.**

**c. Write a program for creating and nesting components (function and class).**

### Experiment 5(a): Rendering HTML to a Web Page

#### 1. Aim of the Program

To create a fundamental React application that renders raw HTML elements directly onto a web page using React DOM methods.

#### 2. Description of the Program

In React, elements are rendered into a root DOM node using react-dom/client. The createRoot() method takes a target container element from the standard DOM (index.html) and creates a React root to manage rendering. This exercise demonstrates how plain HTML structure can be attached directly to the browser DOM using root.render().

#### 3. Program Code

**File 1: index.html**

**HTML:**

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Render HTML in React</title>
  </head>
  <body>
    <!-- Root Container for React Application -->
    <div id="root"></div>
  </body>
</html>
```

**File 2: index.js**

**JavaScript:**

```
import React from 'react';
import ReactDOM from 'react-dom/client';

// Get reference to the root div element in HTML
const container = document.getElementById('root');
const root = ReactDOM.createRoot(container);

// Render basic HTML content into the root element
root.render(
  <div>
    <h1>Welcome to CSE (AI & ML) Department</h1>
    <p>This is a plain HTML rendering example using React DOM.</p>
    <ul>
      <li>Course: B.Tech</li>
      <li>Specialization: Artificial Intelligence & Machine Learning</li>
      <li>Year: III Year</li>
    </ul>
  </div>
);
```

Output:

Welcome to CSE (AI & ML) Department

This is a plain HTML rendering example using React DOM.

• Course: B.Tech
• Specialization: Artificial Intelligence & Machine Learning
• Year: III Year

**Experiment 5(b): Writing Markup with JSX**

**1. Aim of the Program**

To develop a React application that uses JavaScript XML (JSX) to dynamically embed expressions, combine logic, and apply attributes within markup.

**2. Description of the Program**

JSX (JavaScript XML) is a syntax extension for JavaScript that allows writing HTML-like markup inside JavaScript files. JSX enforces distinct syntax rules:

- Elements must be closed (e.g., <img />, <br />).
- Dynamic data and expressions are embedded using single curly braces {expression}.
- HTML attribute names use camelCase (e.g., class becomes className, onclick becomes onClick).

This program demonstrates displaying dynamic student data and conditional styling using JSX.

**3. Program Code**

**File: App.js**

**JavaScript:**

```
import React from 'react';

function App() {
  // Dynamic JavaScript variables
  const student = {
    name: "AARAV SHARMA",
    rollNo: "22AM1A0501",
    gpa: 8.9,
    isPassed: true,
  };

  const imageSrc = "https://via.placeholder.com/100";

  return (
    <div style={{ fontFamily: 'Arial, sans-serif', padding: '20px' }}>
      <h2 style={{ color: '#2c3e50' }}>Student Profile (JSX Demo)</h2>

      {/* Dynamic Expression Embedding */}
```

```
      <p><strong>Name:</strong> {student.name}</p>
      <p><strong>Roll Number:</strong> {student.rollNo}</p>
      <p><strong>CGPA:</strong> {student.gpa} / 10</p>

      {/* Dynamic Attribute Binding */}
      <img src={imageSrc} alt="Student Avatar" style={{ borderRadius: '50%' }} />

      {/* Embedded Conditional Logic */}
      <div style={{ marginTop: '15px' }}>
        <strong>Academic Status: </strong>
        <span style={{ color: student.isPassed ? 'green' : 'red', fontWeight: 'bold' }}>
          {student.isPassed ? 'PROMOTED' : 'DETAINED'}
        </span>
      </div>
    </div>
  );
}

export default App;
```

**Output:**

**Plaintext:**

Student Profile (JSX Demo)

Name: AARAV SHARMA
Roll Number: 22AM1A0501
CGPA: 8.9 / 10

[ Image: Student Avatar ]

Academic Status: PROMOTED

**Experiment 5(c): Creating and Nesting Components (Function & Class)**

**1. Aim of the Program**

To build a modular React application by creating and nesting both **Functional Components** and **Class Components** to establish a hierarchical UI architecture.

**2. Description of the Program**

Components are independent, reusable pieces of UI in React:

- **Functional Components:** JavaScript functions that return JSX. They represent the modern standard in React development.
- **Class Components:** ES6 classes extending React.Component that require a render() method to return JSX.

In this experiment:

1. A **Class Component** (HeaderClass) renders top-level navigation.
2. A **Functional Component** (CourseCardFunction) renders individual course details.
3. A parent **Functional Component** (App) nests these components together to assemble the page layout.

**3. Program Code**

**File 1: HeaderClass.js (Class Component)**

**JavaScript:**

```
import React, { Component } from 'react';

class HeaderClass extends Component {
  render() {
    return (
      <header style={{ backgroundColor: '#1a365d', color: '#ffffff', padding: '15px' }}>
        <h1 style={{ margin: 0 }}>AI & ML Department Portal</h1>
        <p style={{ margin: 0, fontSize: '14px' }}>Class Component Architecture</p>
      </header>
    );
  }
}

export default HeaderClass;
```

**File 2: CourseCardFunction.js (Functional Component)**

**JavaScript:**

```
import React from 'react';

function CourseCardFunction(props) {
  return (
    <div style={{ border: '1px solid #cbd5e1', borderRadius: '8px', padding: '15px', margin: '10px 0' }}>
      <h3 style={{ margin: '0 0 8px 0', color: '#0f172a' }}>{props.code}: {props.title}</h3>
      <p style={{ margin: '0 0 5px 0' }}><strong>Credits:</strong> {props.credits}</p>
      <p style={{ margin: 0 }}><strong>Instructor:</strong> {props.faculty}</p>
    </div>
  );
}

export default CourseCardFunction;
```

**File 3: App.js (Parent Component with Nesting)**

**JavaScript:**

```
import React from 'react';
import HeaderClass from './HeaderClass';
import CourseCardFunction from './CourseCardFunction';

function App() {
  return (
    <div style={{ maxWidth: '600px', margin: '0 auto', fontFamily: 'sans-serif' }}>
      {/* Nesting Class Component */}
      <HeaderClass />
```

```
      <main style={{ padding: '20px' }}>
        <h2>III B.Tech Enrolled Courses</h2>

        {/* Nesting Functional Components with Props */}
        <CourseCardFunction
          code="AIML301"
          title="Deep Learning & Neural Networks"
          credits="4"
          faculty="Dr. R. K. Verma"
        />

        <CourseCardFunction
          code="AIML302"
          title="Natural Language Processing"
          credits="3"
          faculty="Prof. S. Mehta"
        />
      </main>
    </div>
  );
}

export default App;
```

**Output:**

**Plaintext:**

```
AI & ML Department Portal
Class Component Architecture
```

III B.Tech Enrolled Courses

| AIML301: Deep Learning & Neural Networks |

| Credits: 4 |

| Instructor: Dr. R. K. Verma |

| AIML302: Natural Language Processing |

| Credits: 3 |

| Instructor: Prof. S. Mehta |

## 6. ReactJS – Props and States, Styles, Respond to Events

**a. Write a program to work with props and states.**

**b. Write a program to add styles (CSS & Sass Styling) and display data.**

**c. Write a program for responding to events.**

### .a. Program to Work with Props and States

**Aim of the Program**

To design and develop a React application that demonstrates data transfer between components using **Props** and state management within a component using the **useState Hook**.

**Description of the Program**

- **Props (Properties):** Read-only data passed from a parent component (App) to a child component (UserProfile). They make components reusable.
- **State (useState):** Internal component data that can change over time. In this program, a counter state (likeCount) tracks user engagement, causing React to re-render the UI whenever updated.

**Program Code**

JavaScript:

```
// UserProfile.jsx (Child Component displaying Props & managing local State)

import React, { useState } from 'react';

function UserProfile(props) {
```

```
  // State initialization using Hook
  const [likeCount, setLikeCount] = useState(0);

  const handleLike = () => {
    setLikeCount(likeCount + 1);
  };

  return (
    <div style={{ border: '2px solid #007bff', padding: '15px', borderRadius: '8px', width: '300px' }}>
      {/* Accessing Props passed from Parent */}
      <h3>Student Profile</h3>
      <p><strong>Name:</strong> {props.name}</p>
      <p><strong>Roll No:</strong> {props.rollNo}</p>
      <p><strong>Specialization:</strong> {props.branch}</p>

      <hr />

      {/* State implementation */}
      <p><strong>Profile Likes:</strong> {likeCount}</p>
      <button onClick={handleLike} style={{ cursor: 'pointer', padding: '8px 12px' }}>
        👍 Like Profile
      </button>
    </div>
  );
}

// App.jsx (Parent Component)
export default function App() {
  return (
    <div style={{ padding: '20px' }}>
      <h2>6.a - React Props and State Demonstration</h2>
```

```
      {/* Passing data to child component via props */}
      <UserProfile
        name="Aarav Sharma"
        rollNo="22A91A6601"
        branch="CSE (AI & ML)"
      />
    </div>
  );
}
```

**Output**

- **Initial View:** Renders a card titled **Student Profile** displaying details (Aarav Sharma, Roll No, CSE (AI & ML)) with **Profile Likes: 0**.
- **Interaction:** Clicking the **" 👍 Like Profile"** button updates likeCount dynamically without refreshing the page (e.g., updates to **Profile Likes: 1**, **Profile Likes: 2**).

**6.b. Program to Add Styles (CSS & Sass Styling) and Display Data**

**Aim of the Program**

To implement external CSS styling, CSS Modules, and Sass/SCSS preprocessors to structure and display formatted JSON/dynamic data in a React Application.

**Description of the Program**

This application renders an **AI Model Performance Summary** table. It uses:

1. **Sass/SCSS (Card.module.scss):** Utilizes nested rules, variables, and scoped module styling to prevent CSS global scope pollution.
2. **Data Rendering:** Map operations (.map()) to dynamically construct UI elements from an array of model evaluation metrics.

**Program Code**

**File 1: Card.module.scss** *(Style file)*

***Scss***

```
// Sass variables
$primary-color: #2c3e50;
$accent-color: #27ae60;
$border-color: #e2e8f0;
```

```
.container {
  font-family: Arial, sans-serif;
  max-width: 500px;
  margin: 20px 0;

  .header {
    color: $primary-color;
    border-bottom: 2px solid $accent-color;
    padding-bottom: 5px;
  }

  .dataTable {
    width: 100%;
    border-collapse: collapse;
    margin-top: 15px;

    th, td {
      border: 1px solid $border-color;
      padding: 10px;
      text-align: left;
    }

    th {
      background-color: $primary-color;
      color: white;
    }

    tr:nth-child(even) {
      background-color: #f8f9fa;
    }
  }
```

}

**File 2: ModelStats.jsx** *(Component file)*

**JavaScript:**

```
import React from 'react';
import styles from './Card.module.scss'; // Importing Sass Module

export default function ModelStats() {
  // Dynamic JSON Data Array
  const mlModels = [
    { id: 1, name: 'ResNet-50', task: 'Image Classification', accuracy: '94.2%' },
    { id: 2, name: 'BERT-Base', task: 'NLP Sentiment Analysis', accuracy: '89.6%' },
    { id: 3, name: 'YOLOv8', task: 'Object Detection', accuracy: '91.8%' },
  ];

  return (
    <div className={styles.container}>
      <h2 className={styles.header}>AI Model Performance Metrics</h2>
      <table className={styles.dataTable}>
        <thead>
          <tr>
            <th>Model ID</th>
            <th>Model Name</th>
            <th>Task</th>
            <th>Accuracy</th>
          </tr>
        </thead>
        <tbody>
          {mlModels.map((model) => (
```

```
            <tr key={model.id}>
              <td>{model.id}</td>
              <td>{model.name}</td>
              <td>{model.task}</td>
              <td>{model.accuracy}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

**Output**

- A clean, styled table with dark blue headers and alternating row shading (light grey/white).
- Metrics for three AI models are rendered inside the structured table.

**6.c. Program for Responding to Events**

**Aim of the Program**

To build a React component that captures and handles various user events, such as button clicks (onClick), form input changes (onChange), and form submission (onSubmit).

**Description of the Program**

This program implements an interactive **Hyperparameter Input Form** for machine learning models:

- **onChange Event:** Captures user typing in input fields and updates the local state in real time.
- **onSubmit Event:** Handles form submission, triggers e.preventDefault() to avoid page reloads, and displays an alert with user parameters.

**Program Code**

**JavaScript:**

```
import React, { useState } from 'react';

export default function EventHandlingDemo() {
```

```
  // State holding form data
  const [formData, setFormData] = useState({
    learningRate: '0.001',
    epochs: '50',
  });

  // Event handler for typing input (onChange)
  const handleInputChange = (event) => {
    const { name, value } = event.target;
    setFormData((prevData) => ({
      ...prevData,
      [name]: value,
    }));
  };

  // Event handler for form submission (onSubmit)
  const handleSubmit = (event) => {
    event.preventDefault(); // Prevents default browser reload
    alert(`Hyperparameters Saved:\nLearning Rate: ${formData.learningRate}\nEpochs:
${formData.epochs}`);
  };

  return (
    <div style={{ padding: '20px', maxWidth: '400px', fontFamily: 'sans-serif' }}>
      <h2>6.c - Event Handling in React</h2>

      <form onSubmit={handleSubmit}>
        <div style={{ marginBottom: '12px' }}>
          <label>Learning Rate: </label>
          <input
            type="text"
            name="learningRate"
```

```
          value={formData.learningRate}
          onChange={handleInputChange}
          style={{ width: '100%', padding: '8px', marginTop: '4px' }}
        />
      </div>

      <div style={{ marginBottom: '12px' }}>
        <label>Epochs: </label>
        <input
          type="number"
          name="epochs"
          value={formData.epochs}
          onChange={handleInputChange}
          style={{ width: '100%', padding: '8px', marginTop: '4px' }}
        />
      </div>

      <button
        type="submit"
        style={{ padding: '10px 15px', backgroundColor: '#28a745', color: '#fff', border: 'none',
borderRadius: '4px', cursor: 'pointer' }}
      >
        Train Model
      </button>
    </form>

    {/* Live Preview section showing real-time onChange behavior */}
    <div style={{ marginTop: '20px', padding: '10px', background: '#f0f0f0' }}>
      <h4>Live Configuration Preview:</h4>
      <p>Learning Rate: <code>{formData.learningRate}</code></p>
      <p>Epochs: <code>{formData.epochs}</code></p>
    </div>
```

```
    </div>
  );
}
```

**Output**

- **Live Updates:** Typing numbers into the **Learning Rate** or **Epochs** field dynamically updates the **Live Configuration Preview** text instantly via onChange.
- **Alert Box:** Clicking **"Train Model"** triggers onSubmit, intercepting the submit action to pop up a browser dialog reading:

Plaintext:

Hyperparameters Saved:

Learning Rate: 0.001

Epochs: 50

## 7. ReactJS – Conditional Rendering, Rendering Lists, React Forms

**a. Write a program for conditional rendering.**

**b. Write a program for rendering lists.**

**c. Write a program for working with different form fields using react forms.**

### Program 7(a): Conditional Rendering in React

**Aim of the Program**

To write and demonstrate a ReactJS program that conditionally renders different UI components or elements based on application state (e.g., user authentication status).

**Description of the Program**

Conditional rendering in React works the same way conditions work in JavaScript. We use JavaScript operators like if statements, ternary operators (condition ? true : false), or logical AND (&&) operators to create elements representing the current state, letting React update the UI to match them.

In this program, a LoginControl component dynamically displays either a **Welcome Dashboard** or a **Login Request** message along with a button that toggles the user's logged-in state.

**Program Code**

**JavaScript:**

```
// App.jsx
import React, { useState } from 'react';

// User Greeting Component
function UserGreeting({ username }) {
  return (
    <div style={{ color: '#2e7d32', backgroundColor: '#e8f5e9', padding: '15px', borderRadius: '5px'
}}>
      <h2>Welcome back, {username}! 👋</h2>
      <p>You are logged into the AI & ML Student Portal.</p>
    </div>
  );
}

// Guest Greeting Component
function GuestGreeting() {
  return (
    <div style={{ color: '#c62828', backgroundColor: '#ffebee', padding: '15px', borderRadius: '5px' }}>
      <h2>Please Sign In 🔒</h2>
      <p>Access restricted to authorized personnel.</p>
    </div>
  );
}

// Main Conditional Rendering Component
export default function ConditionalRenderDemo() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);
  const [user] = useState('Alex');

  const handleToggle = () => {
```

```
    setIsLoggedIn((prev) => !prev);
  };

  return (
    <div style={{ fontFamily: 'Arial, sans-serif', maxWidth: '400px', margin: '30px auto', textAlign: 'center' }}>
      <h3>Lab 7(a): Conditional Rendering</h3>

      {/* 1. Ternary Operator Rendering */}
      {isLoggedIn ? <UserGreeting username={user} /> : <GuestGreeting />}

      {/* 2. Logical AND (&&) Operator Rendering */}
      {isLoggedIn && (
        <p style={{ fontSize: '12px', color: '#666' }}>
          Session status: Active | System time: {new Date().toLocaleTimeString()}
        </p>
      )}

      {/* Action Button */}
      <button
        onClick={handleToggle}
        style={{
          marginTop: '20px',
          padding: '10px 20px',
          fontSize: '16px',
          cursor: 'pointer',
          backgroundColor: isLoggedIn ? '#d32f2f' : '#1976d2',
          color: '#fff',
          border: 'none',
          borderRadius: '4px'
        }}
      >
```

```
      {isLoggedIn ? 'Log Out' : 'Log In'}
    </button>
  </div>
 );
}
```

**Output**

- **Initial State (Logged Out):**
  - Displays red banner: **"Please Sign In 🔒 "**
  - Displays blue button labeled **Log In**.
- **After clicking Log In:**
  - Displays green banner: **"Welcome back, Alex! 👋 "**
  - Displays active session timestamp below the banner.
  - Button color changes to red and labels **Log Out**.

**Program 7(b): Rendering Lists in React**

**Aim of the Program**

To build a ReactJS application that dynamically transforms arrays of data into structured HTML list items using JavaScript's map() method, incorporating unique key props.

**Description of the Program**

In React, you render lists of elements by traversing array data using the .map() method. Each list item generated dynamically requires a unique key prop (usually an ID). React uses keys to identify which items have changed, been added, or been removed, which optimizes re-rendering performance.

This program displays a list of Machine Learning models along with their primary tasks and status flags.

**Program Code**

**JavaScript:**

```
// App.jsx
import React from 'react';

export default function ListRenderingDemo() {
  // Array of objects representing datasets/models
  const mlModels = [
```

```
  { id: 'm1', name: 'Convolutional Neural Network', category: 'Computer Vision', accuracy: '94.5%' },
  { id: 'm2', name: 'Transformer (BERT/GPT)', category: 'NLP', accuracy: '98.1%' },
  { id: 'm3', name: 'Random Forest', category: 'Classification', accuracy: '89.2%' },
  { id: 'm4', name: 'K-Means Clustering', category: 'Unsupervised', accuracy: 'N/A' },
 ];

 return (
  <div style={{ fontFamily: 'Arial, sans-serif', maxWidth: '500px', margin: '30px auto' }}>
   <h2>Lab 7(b): AI/ML Models Registry</h2>
   <p style={{ color: '#555' }}>Rendering dynamic lists using <code>Array.map()</code> with keys:</p>

   <ul style={{ listStyleType: 'none', padding: 0 }}>
    {mlModels.map((model) => (
     <li
      key={model.id} // Unique Key Requirement
      style={{
       border: '1px solid #ddd',
       borderRadius: '6px',
       padding: '12px 16px',
       marginBottom: '10px',
       boxShadow: '0 2px 4px rgba(0,0,0,0.05)',
       display: 'flex',
       justifyContent: 'space-between',
       alignItems: 'center'
      }}
     >
      <div>
       <strong style={{ fontSize: '16px', color: '#1a237e' }}>{model.name}</strong>
       <div style={{ fontSize: '13px', color: '#666', marginTop: '4px' }}>
        Category: <em>{model.category}</em>
       </div>
```

```
          </div>
          <span style={{
            backgroundColor: '#e3f2fd',
            color: '#0d47a1',
            padding: '4px 8px',
            borderRadius: '12px',
            fontSize: '12px',
            fontWeight: 'bold'
          }}>
            Acc: {model.accuracy}
          </span>
        </li>
      ))}
    </ul>
  </div>
 );
}
```

**Output**

Displays a formatted card list of 4 ML models:

1. **Convolutional Neural Network** | Category: *Computer Vision* | Acc: 94.5%
2. **Transformer (BERT/GPT)** | Category: *NLP* | Acc: 98.1%
3. **Random Forest** | Category: *Classification* | Acc: 89.2%
4. **K-Means Clustering** | Category: *Unsupervised* | Acc: N/A

**Program 7(c): Working with React Forms & Form Fields**

**Aim of the Program**

To construct a controlled React form component handling multiple input elements (text, select dropdown, checkbox, radio buttons) and managing state updates using unified handlers.

**Description of the Program**

In React, form fields whose values are controlled by React state are called **Controlled Components**. Instead of relying on the DOM to hold form data, state variables serve as the "single source of truth." When a user updates a form field, an onChange event updates the state, keeping the input display and component state in sync.

This program captures student project details for an AI project submission portal using various HTML form inputs.

**Program Code**

**JavaScript:**

```
// App.jsx
import React, { useState } from 'react';

export default function ReactFormsDemo() {
  // Centralized form state object
  const [formData, setFormData] = useState({
    studentName: '',
    projectType: 'Computer Vision',
    framework: 'PyTorch',
    agreeToTerms: false
  });

  const [submittedData, setSubmittedData] = useState(null);

  // Unified event handler for all input fields
  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setFormData((prevData) => ({
      ...prevData,
      [name]: type === 'checkbox' ? checked : value
    }));
  };

  const handleSubmit = (e) => {
    e.preventDefault(); // Prevent page reload
    if (!formData.agreeToTerms) {
```

```
    alert('Please agree to the code of conduct before submitting.');
    return;
  }
  setSubmittedData(formData);
};

 return (
  <div style={{ fontFamily: 'Arial, sans-serif', maxWidth: '480px', margin: '30px auto', padding: '20px', border: '1px solid #ccc', borderRadius: '8px' }}>
    <h2>Lab 7(c): AI Project Registration</h2>

    <form onSubmit={handleSubmit}>
      {/* Text Input */}
      <div style={{ marginBottom: '15px' }}>
        <label style={{ display: 'block', marginBottom: '5px', fontWeight: 'bold' }}>Student Name:</label>
        <input
          type="text"
          name="studentName"
          value={formData.studentName}
          onChange={handleChange}
          placeholder="Enter full name"
          required
          style={{ width: '100%', padding: '8px', boxSizing: 'border-box' }}
        />
      </div>

      {/* Dropdown / Select Input */}
      <div style={{ marginBottom: '15px' }}>
        <label style={{ display: 'block', marginBottom: '5px', fontWeight: 'bold' }}>Project Domain:</label>
        <select
```

```
          name="projectType"
          value={formData.projectType}
          onChange={handleChange}
          style={{ width: '100%', padding: '8px' }}
        >
          <option value="Computer Vision">Computer Vision</option>
          <option value="Natural Language Processing">Natural Language Processing</option>
          <option value="Reinforcement Learning">Reinforcement Learning</option>
          <option value="Generative AI">Generative AI</option>
        </select>
      </div>

      {/* Radio Buttons */}
      <div style={{ marginBottom: '15px' }}>
        <label style={{ display: 'block', marginBottom: '5px', fontWeight: 'bold' }}>Primary Framework:</label>
        <label style={{ marginRight: '15px' }}>
          <input
            type="radio"
            name="framework"
            value="PyTorch"
            checked={formData.framework === 'PyTorch'}
            onChange={handleChange}
          /> PyTorch
        </label>
        <label style={{ marginRight: '15px' }}>
          <input
            type="radio"
            name="framework"
            value="TensorFlow"
            checked={formData.framework === 'TensorFlow'}
            onChange={handleChange}
```

```
          /> TensorFlow
        </label>
        <label>
          <input
            type="radio"
            name="framework"
            value="Scikit-Learn"
            checked={formData.framework === 'Scikit-Learn'}
            onChange={handleChange}
          /> Scikit-Learn
        </label>
      </div>

      {/* Checkbox Input */}
      <div style={{ marginBottom: '20px' }}>
        <label>
          <input
            type="checkbox"
            name="agreeToTerms"
            checked={formData.agreeToTerms}
            onChange={handleChange}
          /> I accept the Academic Honor Code
        </label>
      </div>

      {/* Submit Button */}
      <button
        type="submit"
        style={{ width: '100%', padding: '10px', backgroundColor: '#2e7d32', color: 'white', border: 'none', borderRadius: '4px', cursor: 'pointer', fontSize: '16px' }}
      >
        Submit Application
```

```
        </button>
      </form>

      {/* Submitted Results Display */}
      {submittedData && (
        <div style={{ marginTop: '25px', padding: '15px', backgroundColor: '#f0f4c3', borderRadius:
'5px' }}>
          <h4 style={{ margin: '0 0 10px 0', color: '#33691e' }}>Submission Summary:</h4>
          <p style={{ margin: '4px 0' }}><strong>Name:</strong> {submittedData.studentName}</p>
          <p style={{ margin: '4px 0' }}><strong>Domain:</strong> {submittedData.projectType}</p>
          <p style={{ margin: '4px 0' }}><strong>Framework:</strong> {submittedData.framework}</p>
        </div>
      )}
    </div>
  );
}
```

**Output**

1. **User Interface View:**
   - Text field for **Student Name**.
   - Dropdown menu for selecting **Project Domain**.
   - Radio options for selecting **Primary Framework** (PyTorch, TensorFlow, Scikit-Learn).
   - Checkbox to accept the **Academic Honor Code**.
   - Green **Submit Application** button.
2. **On Successful Submission:**
   - Generates a yellow summary box at the bottom displaying the selected state choices in real time.

## 8. ReactJS – React Router, Updating the Screen

**a. Write a program for routing to different pages using react router.**

**b. Write a program for updating the screen.**

**Experiment 8(a): Page Routing Using React Router**

**1. Aim of the Program**

To write and demonstrate a ReactJS application that implements client-side routing between different pages (Home, About, Contact) using the react-router-dom library.

**2. Description of the Program**

In single-page applications (SPAs), page reloading is avoided to maintain a fast, seamless user experience. **React Router** provides client-side navigation by mapping specific URL paths to corresponding React components.

- **BrowserRouter**: Standard router component that uses the HTML5 history API to keep UI in sync with the URL.
- **Routes**: Acts as a container for all individual Route definitions.
- **Route**: Renders a specific component based on the URL path (path="/...").
- **Link**: Replaces standard HTML <a> tags to allow seamless navigation without refreshing the browser page.

**3. Prerequisites & Installation**

Before writing the code, install react-router-dom inside your React project directory:

**Bash:**

```
npm install react-router-dom
```

**4. Program Code**

**File: src/App.js**

**JavaScript:**

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route, Link } from 'react-router-dom';

// 1. Component Definitions for different pages
function Home() {
  return (
```

```
    <div style={pageStyle}>
      <h2>🏠 Home Page</h2>
      <p>Welcome to the Department of CSE (AI & ML) Portal!</p>
    </div>
  );
}

function About() {
  return (
    <div style={pageStyle}>
      <h2>ℹ️ About Us Page</h2>
      <p>This course focuses on Artificial Intelligence and Machine Learning applications in modern Web Development.</p>
    </div>
  );
}

function Contact() {
  return (
    <div style={pageStyle}>
      <h2>📞 Contact Page</h2>
      <p>Reach out to the Dept of CSE (AI & ML) at: <code>cse.aiml@college.edu</code></p>
    </div>
  );
}

// 2. Main App Component displaying Navbar and Route configuration
export default function App() {
  return (
    <Router>
      <div style={{ fontFamily: 'Arial, sans-serif', textAlign: 'center', padding: '20px' }}>
        <header>
```

```
        <h1>React Router Demonstration</h1>
        {/* Navigation Bar */}
        <nav style={navStyle}>
          <Link to="/" style={linkStyle}>Home</Link>
          <Link to="/about" style={linkStyle}>About</Link>
          <Link to="/contact" style={linkStyle}>Contact</Link>
        </nav>
      </header>

      <hr style={{ margin: '20px 0' }} />

      {/* Dynamic Route Switching */}
      <main>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
          <Route path="/contact" element={<Contact />} />
        </Routes>
      </main>
    </div>
  </Router>
 );
}

// Styling Objects
const navStyle = {
 backgroundColor: '#282c34',
 padding: '10px',
 borderRadius: '8px',
 display: 'inline-block'
};
```

```
const linkStyle = {
  color: '#61dafb',
  margin: '0 15px',
  textDecoration: 'none',
  fontSize: '18px',
  fontWeight: 'bold'
};

const pageStyle = {
  padding: '20px',
  border: '1px solid #ccc',
  borderRadius: '8px',
  backgroundColor: '#f9f9f9',
  maxWidth: '500px',
  margin: '0 auto'
};
```

**5. Output**

**Initial View (http://localhost:3000/)**

**Plaintext:**

React Router Demonstration

[ Home ] [ About ] [ Contact ]

🏠 Home Page

Welcome to the Department of CSE (AI & ML) Portal!

Upon clicking 'About' (http://localhost:3000/about)

**Plaintext:**

React Router Demonstration

[ Home ] [ About ] [ Contact ]

---

**i** About Us Page

This course focuses on Artificial Intelligence

and Machine Learning applications in modern Web Dev.

**Experiment 8(b): Updating the Screen Using React State (useState)**

**1. Aim of the Program**

To write a ReactJS application that dynamically updates content on the screen in response to user events using the useState Hook.

**2. Description of the Program**

In React, components update what is displayed on the screen using **State**. When state data changes, React automatically re-renders the component to reflect the new state in the DOM.

- **useState Hook**: A special function provided by React that lets functional components remember and modify dynamic data.
- **Syntax**: const [stateVariable, setFunction] = useState(initialValue);
- **Triggering Re-renders**: Direct updates to variables do not trigger a render; calling setFunction(...) notifies React to update the UI on the screen.

**3. Program Code**

**File: src/App.js**

**JavaScript:**

```
import React, { useState } from 'react';

export default function CounterApp() {
  // 1. Declare state variables
  const [count, setCount] = useState(0);
  const [step, setStep] = useState(1);
```

```
// 2. Event Handler functions
const handleIncrement = () => setCount(count + step);
const handleDecrement = () => setCount(count - step);
const handleReset = () => {
  setCount(0);
  setStep(1);
};

return (
  <div style={containerStyle}>
    <h2>Dynamic Screen Update Counter</h2>

    {/* Dynamic State Value Rendered on Screen */}
    <div style={displayBoxStyle}>
      <h1 style={{ fontSize: '48px', margin: '10px 0', color: count < 0 ? '#d9534f' : '#5cb85c' }}>
        {count}
      </h1>
      <p>Current Step Size: <strong>{step}</strong></p>
    </div>

    {/* Controls to Update State */}
    <div style={{ margin: '20px 0' }}>
      <button onClick={handleDecrement} style={btnDanger}>Decrement (-{step})</button>
      <button onClick={handleIncrement} style={btnSuccess}>Increment (+{step})</button>
    </div>

    <div>
      <label htmlFor="stepInput" style={{ marginRight: '10px' }}>Adjust Step Size:</label>
      <input
        id="stepInput"
        type="number"
```

```
          value={step}
          onChange={(e) => setStep(Number(e.target.value) || 1)}
          style={inputStyle}
        />
      </div>

      <div style={{ marginTop: '20px' }}>
        <button onClick={handleReset} style={btnReset}>Reset All</button>
      </div>
    </div>
  );
}

// Styling Objects
const containerStyle = {
  fontFamily: 'Arial, sans-serif',
  textAlign: 'center',
  maxWidth: '400px',
  margin: '40px auto',
  padding: '20px',
  border: '2px solid #ddd',
  borderRadius: '12px',
  boxShadow: '0 4px 8px rgba(0,0,0,0.1)'
};

const displayBoxStyle = {
  backgroundColor: '#f4f4f9',
  padding: '15px',
  borderRadius: '8px',
  marginBottom: '20px'
};
```

```
const btnStyle = {
  padding: '10px 15px',
  margin: '0 5px',
  fontSize: '16px',
  fontWeight: 'bold',
  color: '#fff',
  border: 'none',
  borderRadius: '5px',
  cursor: 'pointer'
};

const btnSuccess = { ...btnStyle, backgroundColor: '#28a745' };
const btnDanger = { ...btnStyle, backgroundColor: '#dc3545' };
const btnReset = { ...btnStyle, backgroundColor: '#6c757d' };

const inputStyle = {
  width: '60px',
  padding: '5px',
  fontSize: '16px',
  textAlign: 'center'
};
```

**4. Output**

**Initial Screen View**

**Plaintext:**

```
        Dynamic Screen Update Counter
      |            0            |
      |   Current Step Size: 1   |
```

[ Decrement (-1) ]  [ Increment (+1) ]

Adjust Step Size: [ 1 ]

[ Reset All ]

**Screen State After Clicking "Increment" 3 Times**

**Plaintext:**

Dynamic Screen Update Counter

| 3 |

| Current Step Size: 1 |

[ Decrement (-1) ]  [ Increment (+1) ]

Adjust Step Size: [ 1 ]

[ Reset All ]

## 9. ReactJS – Hooks, Sharing data between Components

### a. Write a program to understand the importance of using hooks.

### b. Write a program for sharing data between components.

### Part A: Understanding the Importance of Hooks

**Aim of the Program**

To demonstrate the importance and usage of **React Hooks** (useState and useEffect) in functional components for managing state and handling side effects without writing class components.

**Description of the Program**

Before React 16.8, state management and lifecycle methods were only available in Class Components, which often led to complex and verbose code. **Hooks** allow functional components to attach local state and perform side effects.

In this program:

- **useState Hook**: Manages a simple counter state (count) and updates the UI dynamically when the state changes.
- **useEffect Hook**: Handles side effects by updating the document title whenever the count state updates, mimicking componentDidMount and componentDidUpdate.

**Program Code (App.jsx)**

**JavaScript:**

```
import React, { useState, useEffect } from 'react';

function CounterHooks() {
  // 1. useState Hook: Declares a state variable 'count' initialized to 0
  const [count, setCount] = useState(0);

  // 2. useEffect Hook: Runs side effects when 'count' changes
  useEffect(() => {
    document.title = `Clicked ${count} times`;
  }, [count]); // Dependency array ensures it only runs when 'count' changes

  return (
    <div style={{ textAlign: 'center', marginTop: '50px', fontFamily: 'Arial, sans-serif' }}>
      <h2>React Hooks Demonstration</h2>
      <p style={{ fontSize: '20px' }}>Current Count: <strong>{count}</strong></p>

      <button
        onClick={() => setCount(count + 1)}
        style={{ padding: '10px 20px', fontSize: '16px', margin: '5px', cursor: 'pointer' }}
      >
        Increment
      </button>

      <button
```

```
        onClick={() => setCount(count - 1)}
        style={{ padding: '10px 20px', fontSize: '16px', margin: '5px', cursor: 'pointer' }}
      >
        Decrement
      </button>

      <button
        onClick={() => setCount(0)}
        style={{ padding: '10px 20px', fontSize: '16px', margin: '5px', cursor: 'pointer' }}
      >
        Reset
      </button>
    </div>
  );
}

export default CounterHooks;
```

**Output**

When the application runs in the browser:

1. **Initial Screen:**
    - Text displays: Current Count: 0
    - Browser tab title shows: Clicked 0 times
2. **Action (Click 'Increment' twice):**
    - Text updates instantly to: Current Count: 2
    - Browser tab title updates to: Clicked 2 times
3. **Action (Click 'Reset'):**
    - Text resets to: Current Count: 0

**Part B: Sharing Data Between Components**

**Aim of the Program**

To construct a React application that demonstrates sharing data between components using **Props** (Parent to Child) and **Callback Functions** (Child to Parent).

**Description of the Program**

In React, data flows top-down (unidirectional) from Parent to Child via **Props**. When a Child component needs to update or pass data back to a Parent, the Parent passes down a **handler function** as a prop, which the Child component invokes with new data.

In this program:

- **ParentComponent**: Holds the shared state (userMessage) and defines an update handler.
- **ChildDisplay**: Receives data via props from Parent and renders it.
- **ChildInput**: Receives a function via props from Parent and updates Parent's state when text is typed into an input field.

**Program Code (DataSharingApp.jsx)**

**JavaScript:**

```
import React, { useState } from 'react';

// Child Component 1: Receives data via props and displays it
function ChildDisplay({ message }) {
  return (
    <div style={{ border: '2px dashed #007bff', padding: '15px', marginBottom: '15px' }}>
      <h3>Child Component A (Display)</h3>
      <p>Received from Parent: <strong>{message || "No message entered yet."}</strong></p>
    </div>
  );
}

// Child Component 2: Receives a function prop to send data back to Parent
function ChildInput({ onMessageChange }) {
  return (
    <div style={{ border: '2px dashed #28a745', padding: '15px' }}>
      <h3>Child Component B (Input)</h3>
      <label htmlFor="userInput">Type to share data: </label>
```

```
      <input
        id="userInput"
        type="text"
        placeholder="Type something..."
        onChange={(e) => onMessageChange(e.target.value)}
        style={{ padding: '8px', width: '250px' }}
      />
    </div>
  );
}

// Parent Component: Holds state and manages data flow
export default function ParentContainer() {
  const [sharedText, setSharedText] = useState("");

  // Callback function to handle data sent from Child B
  const handleTextUpdate = (newText) => {
    setSharedText(newText);
  };

  return (
    <div style={{ maxWidth: '500px', margin: '40px auto', fontFamily: 'Arial, sans-serif' }}>
      <h2>Parent Component</h2>
      <p style={{ backgroundColor: '#f4f4f4', padding: '10px' }}>
        Parent's Shared State: <em>{sharedText}</em>
      </p>

      {/* Passing data down to Child A */}
      <ChildDisplay message={sharedText} />

      {/* Passing callback function down to Child B */}
```

```
      <ChildInput onMessageChange={handleTextUpdate} />
    </div>
  );
}
```

**Output**

When the application loads:

1. **Initial View:**
    - Parent state displays empty text.
    - **Child A (Display)** shows: Received from Parent: No message entered yet.
    - **Child B (Input)** renders an input box.
2. **User Interaction:**
    - The user types "Hello React Hooks" into the input field in **Child B**.
    - **Child B** invokes onMessageChange("Hello React Hooks").
    - **Parent Container** updates its state sharedText.
    - **Child A (Display)** re-renders immediately to show: Received from Parent: Hello React Hooks

## 10. MongoDB – Installation, Configuration, CRUD operations

## a. Install MongoDB and configure ATLAS

## b. Write MongoDB queries to perform CRUD operations on document using insert(), find(), update(), remove()

### 1. AIM OF THE PROGRAM

To successfully set up a MongoDB Atlas cloud database instance, configure connection parameters, and write MongoDB shell queries to execute basic CRUD (Create, Read, Update, Delete) operations on JSON documents using standard methods: insertOne(), insertMany(), find(), updateOne(), updateMany(), deleteOne(), and deleteMany().

### 2. DESCRIPTION OF THE PROGRAM

**MongoDB** is an open-source, high-performance, document-oriented NoSQL database system designed for high availability and dynamic scaling. Unlike traditional RDBMS databases that store data in tables with rigid rows and columns, MongoDB stores data in flexible, schema-free collections containing **BSON** (Binary JSON) documents.

**Core MongoDB Concepts vs. RDBMS**

| RDBMS Concept | MongoDB Concept | Description |
|---|---|---|
| Database | Database | Container for collections. |
| Table | Collection | Group of MongoDB documents. |
| Row | Document | A single record consisting of field-value pairs. |
| Column | Field | Key-value pair inside a document. |
| Primary Key | _id | Automatically generated unique identifier (ObjectId). |

**Step-by-Step Atlas Configuration**

1. **Create an Account:** Register on MongoDB Atlas.
2. **Build a Cluster:** Select the free **M0 Shared Cluster** tier and choose a region (e.g., AWS / Mumbai).
3. **Network Access:** Under Network Access, add IP address 0.0.0.0/0 to allow access from any IP during lab practice.
4. **Database Access:** Create a database user (e.g., Username: student, Password: LabPassword123).
5. **Connection String:** Obtain the connection URI:

mongodb+srv://student:<password>@cluster0.mongodb.net/

6. **Access Method:** Open MongoDB Shell (mongosh) or MongoDB Compass to execute CRUD operations against the cloud cluster or a local MongoDB server instance.

**3. CODE (MongoDB Shell Script / Commands)**

Execute the following commands in the MongoDB Shell (mongosh) or MongoDB Compass console:

**JavaScript:**

```
//STEP 1: DATABASE & COLLECTION SELECTION

Switch to (or create) database named 'aiml_lab'
use aiml_lab;

Clear collection if it already exists
db.students.drop();
```

```
//STEP 2: CREATE OPERATIONS (INSERT)

 Insert a single document
db.students.insertOne({
   rollNo: 101,
   name: "Aarav Sharma",
   branch: "AI & ML",
   year: 3,
   cgpa: 8.5,
   skills: ["Python", "TensorFlow", "MongoDB"]
});

// Insert multiple documents at once
db.students.insertMany([
   {
      rollNo: 102,
      name: "Priya Patel",
      branch: "AI & ML",
      year: 3,
      cgpa: 9.1,
      skills: ["Java", "PyTorch", "SQL"]
   },
   {
      rollNo: 103,
      name: "Rohan Verma",
      branch: "CSE",
      year: 2,
      cgpa: 7.4,
      skills: ["C++", "Python"]
   },
```

```
    {
        rollNo: 104,
        name: "Ananya Iyer",
        branch: "AI & ML",
        year: 3,
        cgpa: 8.9,
        skills: ["R", "Machine Learning", "Data Analysis"]
    }
]);


//STEP 3: READ OPERATIONS (FIND)

// Fetch all documents in formatted (pretty) view
db.students.find();

// Fetch documents with a query filter (branch = 'AI & ML')
db.students.find({ branch: "AI & ML" });

// Fetch document with conditional filter (cgpa > 8.5)
db.students.find({ cgpa: { $gt: 8.5 } });

// Fetch specific fields using projection (1 = show, 0 = hide)
db.students.find(
    { branch: "AI & ML" },
    { _id: 0, name: 1, cgpa: 1 }
);

 //STEP 4: UPDATE OPERATIONS (UPDATE)
 //Update a single document (Increase CGPA of Rohan Verma)
db.students.updateOne(
```

```
    { rollNo: 103 },
    { $set: { cgpa: 7.9, year: 3 } }
);

// Update multiple documents (Add a new field 'status' to all 3rd year students)
db.students.updateMany(
    { year: 3 },
    { $set: { status: "Active" } }
);

// Verify updates
db.students.find({ rollNo: 103 });



// STEP 5: DELETE OPERATIONS (REMOVE)

// Delete a single document (rollNo = 103)
db.students.deleteOne({ rollNo: 103 });

// Delete documents based on dynamic condition (branch != 'AI & ML')
db.students.deleteMany({ branch: { $ne: "AI & ML" } });

// Display final collection state
db.students.find();
```

**4. OUTPUT**

**Output for Step 1 & 2: Database Switch & Insert Operations**

**JSON:**

```
switched to db aiml_lab
```

```
{
  "acknowledged": true,
  "insertedId": ObjectId("665f1a2b9c3e4f0011aa2233")
}

{
  "acknowledged": true,
  "insertedIds": {
    "0": ObjectId("665f1a2b9c3e4f0011aa2234"),
    "1": ObjectId("665f1a2b9c3e4f0011aa2235"),
    "2": ObjectId("665f1a2b9c3e4f0011aa2236")
  }
}
```

**Output for Step 3: Find Operations**

**JSON:**

```
// Query: db.students.find({ branch: "AI & ML" })
[
  {
    "_id": ObjectId("665f1a2b9c3e4f0011aa2233"),
    "rollNo": 101,
    "name": "Aarav Sharma",
    "branch": "AI & ML",
    "year": 3,
    "cgpa": 8.5,
    "skills": [ "Python", "TensorFlow", "MongoDB" ]
  },
  {
```

```
    "_id": ObjectId("665f1a2b9c3e4f0011aa2234"),
    "rollNo": 102,
    "name": "Priya Patel",
    "branch": "AI & ML",
    "year": 3,
    "cgpa": 9.1,
    "skills": [ "Java", "PyTorch", "SQL" ]
  },
  {
    "_id": ObjectId("665f1a2b9c3e4f0011aa2236"),
    "rollNo": 104,
    "name": "Ananya Iyer",
    "branch": "AI & ML",
    "year": 3,
    "cgpa": 8.9,
    "skills": [ "R", "Machine Learning", "Data Analysis" ]
  }
]

// Query with Projection: db.students.find({ branch: "AI & ML" }, { _id: 0, name: 1, cgpa: 1 })
[
  { "name": "Aarav Sharma", "cgpa": 8.5 },
  { "name": "Priya Patel", "cgpa": 9.1 },
  { "name": "Ananya Iyer", "cgpa": 8.9 }
]
```


**Output for Step 4: Update Operations**

**JSON:**


```
// updateOne Result:
```

```
{
  "acknowledged": true,
  "insertedId": null,
  "matchedCount": 1,
  "modifiedCount": 1,
  "upsertedCount": 0
}

// updateMany Result:
{
  "acknowledged": true,
  "insertedId": null,
  "matchedCount": 4,
  "modifiedCount": 4,
  "upsertedCount": 0
}
```

**Output for Step 5: Delete Operations & Final Collection State**

**JSON:**

```
// deleteOne Result:
{
  "acknowledged": true,
  "deletedCount": 1
}

// Final Collection Query output: db.students.find()
[
  {
    "_id": ObjectId("665f1a2b9c3e4f0011aa2233"),
```

```
  "rollNo": 101,
  "name": "Aarav Sharma",
  "branch": "AI & ML",
  "year": 3,
  "cgpa": 8.5,
  "skills": [ "Python", "TensorFlow", "MongoDB" ],
  "status": "Active"
 },
 {
  "_id": ObjectId("665f1a2b9c3e4f0011aa2234"),
  "rollNo": 102,
  "name": "Priya Patel",
  "branch": "AI & ML",
  "year": 3,
  "cgpa": 9.1,
  "skills": [ "Java", "PyTorch", "SQL" ],
  "status": "Active"
 },
 {
  "_id": ObjectId("665f1a2b9c3e4f0011aa2236"),
  "rollNo": 104,
  "name": "Ananya Iyer",
  "branch": "AI & ML",
  "year": 3,
  "cgpa": 8.9,
  "skills": [ "R", "Machine Learning", "Data Analysis" ],
  "status": "Active"
 }
]
```

**RESULT**

MongoDB cluster on MongoDB Atlas was created successfully, and CRUD operations (insertOne, insertMany, find, updateOne, updateMany, deleteOne, and deleteMany) were executed using the MongoDB Shell.

## 11. MongoDB – Databases, Collections and Records

**a. Write MongoDB queries to Create and drop databases and collections.**

**b. Write MongoDB queries to work with records using find(), limit(), sort(), createIndex(), aggregate().**

**AIM OF THE PROGRAM**

1. To demonstrate creation and deletion of MongoDB databases and collections using MongoDB Shell (mongosh) commands.
2. To manipulate and query records (documents) using MongoDB methods: find(), limit(), sort(), createIndex(), and aggregate().

**DESCRIPTION OF THE PROGRAM**

**Part A: Database & Collection Operations**

MongoDB is a document-oriented NoSQL database.

- **Database**: Holds collections of documents. In MongoDB, a database is created implicitly when data is inserted into it. Dropping a database permanently removes all underlying data.
- **Collection**: A grouping of MongoDB documents (analogous to a table in relational databases). Collections are created explicitly via createCollection() or implicitly upon document insertion.

**Part B: Record Manipulation & Operations**

Documents in MongoDB are stored as JSON-like BSON (Binary JSON) objects.

- **find()**: Retrieves documents matching specified query criteria.
- **limit()**: Restricts the maximum number of documents returned.
- **sort()**: Orders returned documents in ascending (1) or descending (-1) order.
- **createIndex()**: Creates indexes on fields to improve query execution speed.
- **aggregate()**: Processes data through a multi-stage pipeline (e.g., filtering with $match, grouping with $group) to compute aggregated results.

**CODE (MongoDB Shell Script)**

**JavaScript:**

```
// PART A: CREATE AND DROP DATABASES AND COLLECTIONS

// 1. Switch to (or implicitly create) a new database named 'aiml_db'
use aiml_db;

// 2. Explicitly create a collection named 'students'
db.createCollection("students");

// 3. Verify collection creation
show collections;

// 4. (Optional Cleanup Demo) Drop a temporary collection
db.createCollection("temp_data");
db.temp_data.drop();

// Note: 'db.dropDatabase()' can be used to remove the active database completely.

// PART B: CRUD AND ADVANCED RECORD OPERATIONS

// --- 1. SETUP: Insert Sample Records ---
db.students.insertMany([
  { student_id: 101, name: "Alice", branch: "AI&ML", year: 3, gpa: 8.8, state: "Telangana" },
  { student_id: 102, name: "Bob", branch: "AI&ML", year: 3, gpa: 7.5, state: "Karnataka" },
  { student_id: 103, name: "Charlie", branch: "CSE", year: 2, gpa: 9.2, state: "Telangana" },
  { student_id: 104, name: "Diana", branch: "AI&ML", year: 3, gpa: 9.5, state: "Maharashtra" },
  { student_id: 105, name: "Ethan", branch: "ECE", year: 3, gpa: 8.1, state: "Telangana" },
  { student_id: 106, name: "Fiona", branch: "AI&ML", year: 2, gpa: 8.9, state: "Karnataka" }
]);

// --- 2. find() ---
// Find all students in the 'AI&ML' branch
```

```
db.students.find({ branch: "AI&ML" });

// --- 3. limit() ---
// Retrieve the first 2 records of AI&ML students
db.students.find({ branch: "AI&ML" }).limit(2);

// --- 4. sort() ---
// Retrieve AI&ML students sorted by GPA in descending order (-1)
db.students.find({ branch: "AI&ML" }).sort({ gpa: -1 });

// Combine sort() and limit(): Get Top 2 students with highest GPA
db.students.find().sort({ gpa: -1 }).limit(2);

// --- 5. createIndex() ---
// Create an ascending index on the 'student_id' field for faster lookups
db.students.createIndex({ student_id: 1 });

// Verify created indexes
db.students.getIndexes();

// --- 6. aggregate() ---
// Pipeline Example: Filter year 3 students, group by branch, and calculate average GPA
db.students.aggregate([
  {
   $match: { year: 3 }
  },
  {
   $group: {
    _id: "$branch",
    average_gpa: { $avg: "$gpa" },
    total_students: { $sum: 1 }
```

```
    }
  },
  {
    $sort: { average_gpa: -1 }
  }
]);
```

**OUTPUT**

**Part A Execution**

**Plaintext:**

```
switched to db aiml_db

{ ok: 1 }

students

true
```

**Part B Execution**

**1. Insert Operation**

**JSON:**

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('65df102a9b1a1a1a1a1a1001'),
    '1': ObjectId('65df102a9b1a1a1a1a1a1002'),
```

```
    '2': ObjectId('65df102a9b1a1a1a1a1a1003'),
    '3': ObjectId('65df102a9b1a1a1a1a1a1004'),
    '4': ObjectId('65df102a9b1a1a1a1a1a1005'),
    '5': ObjectId('65df102a9b1a1a1a1a1a1006')
  }
}
```

**2. Querying with find() (Filter: branch: "AI&ML")**

**JSON:**

```
[
  { _id: ObjectId('...'), student_id: 101, name: 'Alice', branch: 'AI&ML', year: 3, gpa: 8.8, state: 'Telangana' },
  { _id: ObjectId('...'), student_id: 102, name: 'Bob', branch: 'AI&ML', year: 3, gpa: 7.5, state: 'Karnataka' },
  { _id: ObjectId('...'), student_id: 104, name: 'Diana', branch: 'AI&ML', year: 3, gpa: 9.5, state: 'Maharashtra' },
  { _id: ObjectId('...'), student_id: 106, name: 'Fiona', branch: 'AI&ML', year: 2, gpa: 8.9, state: 'Karnataka' }
]
```

**3. Combining find(), sort(), and limit() (Top 2 Highest GPA)**

**JSON:**

```
[
  { _id: ObjectId('...'), student_id: 104, name: 'Diana', branch: 'AI&ML', year: 3, gpa: 9.5, state: 'Maharashtra' },
  { _id: ObjectId('...'), student_id: 103, name: 'Charlie', branch: 'CSE', year: 2, gpa: 9.2, state: 'Telangana' }
]
```

**4. createIndex() and getIndexes() Output**

**JSON:**

student_id_1

```
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { student_id: 1 }, name: 'student_id_1' }
]
```

**5. Aggregation Pipeline Output**

***(Filtering Year 3 students, grouping by branch, and computing average GPA)***

**JSON:**

```
[
  { _id: 'AI&ML', average_gpa: 8.6, total_students: 3 },
  { _id: 'ECE', average_gpa: 8.1, total_students: 1 }
]
```

**12. Augmented Programs: (Any 2 must be completed)**

**a. Design a to-do list application using NodeJS and ExpressJS.**

**b. Design a Quiz app using ReactJS.**

**c. Complete the MongoDB certification from MongoDB University website.**

**EXPERIMENT 12(a): Design a To-Do List Application using Node.js and Express.js**

**AIM OF THE PROGRAM**

To design and implement a persistent, server-side rendered **To-Do List Web Application** using **Node.js** as the runtime environment and **Express.js** as the web framework to demonstrate basic CRUD (Create, Read, Delete) operations and RESTful routing.

**DESCRIPTION OF THE PROGRAM**

The application acts as a task management tool. It sets up an Express server that listens for incoming HTTP requests and maintains an array of task items.

Key features include:

- **GET /**: Renders the main web page displaying the current list of tasks and a submission form.
- **POST /add**: Handles form submission, receives the task payload from the body, appends it to the in-memory array, and redirects back to the homepage.
- **POST /delete**: Removes a specified task item by its index from the list.

It uses embedded HTML forms for interaction, demonstrating key backend web development concepts like request parsing, dynamic HTML construction, routing, and HTTP status management.

**CODE**

**1. File: package.json**

**JSON:**

```
{
  "name": "todo-express-app",
  "version": "1.0.0",
  "main": "server.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

**2. File: server.js**

**JavaScript:**

```
const express = require('express');
const app = express();
const PORT = 3000;

// Middleware to parse urlencoded body data from HTML forms
app.use(express.urlencoded({ extended: true }));

// In-memory data store for tasks
let todoList = [
  "Complete AI & ML Lab Experiments",
```

```
  "Study Express.js Middleware"
];

// 1. GET Route - Render HTML UI & Tasks
app.get('/', (req, res) => {
  let listItemsHTML = todoList.map((task, index) => `
    <li style="margin-bottom: 8px;">
      <span>${task}</span>
      <form action="/delete" method="POST" style="display:inline; margin-left: 10px;">
        <input type="hidden" name="taskIndex" value="${index}" />
        <button type="submit" style="background:#ff4d4d; color:white; border:none; padding:4px 8px; cursor:pointer; border-radius:3px;">Delete</button>
      </form>
    </li>
  `).join('');

  const htmlResponse = `
    <!DOCTYPE html>
    <html>
    <head>
      <title>Express To-Do List</title>
      <style>
        body { font-family: Arial, sans-serif; margin: 40px; background-color: #f4f7f6; }
        .container { max-width: 500px; background: white; padding: 20px; border-radius: 8px; box-shadow: 0 2px 5px rgba(0,0,0,0.1); }
        input[type="text"] { width: 70%; padding: 8px; }
        button.add-btn { padding: 8px 15px; background: #28a745; color: white; border: none; cursor: pointer; border-radius:3px; }
      </style>
    </head>
    <body>
      <div class="container">
```

```
        <h2>B.Tech AI&ML To-Do Manager</h2>
        <form action="/add" method="POST">
          <input type="text" name="newTask" placeholder="Enter a new task..." required />
          <button type="submit" class="add-btn">Add Task</button>
        </form>
        <hr>
        <ul>${listItemsHTML}</ul>
      </div>
    </body>
    </html>
  `;
  res.send(htmlResponse);
});

// 2. POST Route - Add Task
app.post('/add', (req, res) => {
  const task = req.body.newTask;
  if (task) {
    todoList.push(task);
  }
  res.redirect('/');
});

// 3. POST Route - Delete Task
app.post('/delete', (req, res) => {
  const index = req.body.taskIndex;
  if (index !== undefined && index >= 0 && index < todoList.length) {
    todoList.splice(index, 1);
  }
  res.redirect('/');
});
```

```
// Start Server
app.listen(PORT, () => {
  console.log(`To-Do Server running on http://localhost:${PORT}`);
});
```

**OUTPUT**

**Console Output:**

```
> node server.js
To-Do Server running on http://localhost:3000
```

Browser UI Output (http://localhost:3000):

**Plaintext**:

```
  B.Tech AI&ML To-Do Manager
[ Add Task ]
  [ Enter a new task...  ]
  • Complete AI & ML Lab Experiments   [Delete]
  • Study Express.js Middleware        [Delete]
```

**EXPERIMENT 12(b): Design a Quiz Application using React.js**

**AIM OF THE PROGRAM**

To build a dynamic, interactive single-page **Quiz Application** using **React.js** demonstrating functional components, JSX syntax, React state management (useState), event handling, and conditional rendering.

**DESCRIPTION OF THE PROGRAM**

The application displays a series of multiple-choice questions one at a time. It manages user progress, maintains the current score dynamically without refreshing the page, and tracks question indexes using React state hooks.

Key features:

- **State Management**: Uses useState to monitor currentQuestionIndex, user score, and showScore state boolean.

- **Option Handling**: Evaluates user selections instantly and updates total marks earned.
- **Results View**: Once all questions are completed, conditionally renders a final summary card displaying the final score alongside a **Restart Quiz** action button.

**CODE**

**1. File: src/App.js**

**JavaScript:**

```
import React, { useState } from 'react';

const quizData = [
  {
    questionText: 'Which algorithm is commonly used for classification in Machine Learning?',
    options: ['Linear Regression', 'Support Vector Machines (SVM)', 'K-Means Clustering', 'PCA'],
    answer: 'Support Vector Machines (SVM)'
  },
  {
    questionText: 'Which activation function outputs values in the range (0, 1)?',
    options: ['ReLU', 'Tanh', 'Sigmoid', 'Leaky ReLU'],
    answer: 'Sigmoid'
  },
  {
    questionText: 'Node.js is built on which JavaScript Engine?',
    options: ['SpiderMonkey', 'V8', 'Chakra', 'JavaScriptCore'],
    answer: 'V8'
  }
];

export default function App() {
  const [currentQuestion, setCurrentQuestion] = useState(0);
  const [score, setScore] = useState(0);
  const [showScore, setShowScore] = useState(false);
```

```
const handleAnswerOptionClick = (selectedOption) => {
  if (selectedOption === quizData[currentQuestion].answer) {
    setScore(prevScore => prevScore + 1);
  }

  const nextQuestion = currentQuestion + 1;
  if (nextQuestion < quizData.length) {
    setCurrentQuestion(nextQuestion);
  } else {
    setShowScore(true);
  }
};

const resetQuiz = () => {
  setCurrentQuestion(0);
  setScore(0);
  setShowScore(false);
};

return (
  <div style={styles.body}>
    <div style={styles.card}>
      {showScore ? (
        <div style={styles.scoreSection}>
          <h2>Quiz Complete!</h2>
          <p style={{ fontSize: '18px' }}>
            You scored <strong>{score}</strong> out of <strong>{quizData.length}</strong>
          </p>
          <button style={styles.button} onClick={resetQuiz}>Restart Quiz</button>
        </div>
```

```
      ) : (
        <div>
          <div style={styles.header}>
            <h3>Question {currentQuestion + 1} / {quizData.length}</h3>
          </div>
          <div style={styles.questionText}>
            {quizData[currentQuestion].questionText}
          </div>
          <div style={styles.optionsContainer}>
            {quizData[currentQuestion].options.map((option, idx) => (
              <button
                key={idx}
                style={styles.optionButton}
                onClick={() => handleAnswerOptionClick(option)}
              >
                {option}
              </button>
            ))}
          </div>
        </div>
      )}
    </div>
  </div>
 );
}

// Inline Styles for React Component
const styles = {
  body: {
    display: 'flex',
    justifyContent: 'center',
```

```
  alignItems: 'center',
  height: '100vh',
  backgroundColor: '#282c34',
  fontFamily: 'Arial, sans-serif'
},
card: {
  backgroundColor: '#fff',
  padding: '30px',
  borderRadius: '10px',
  width: '450px',
  boxShadow: '0 4px 10px rgba(0, 0, 0, 0.3)',
  textAlign: 'center'
},
header: { marginBottom: '15px', color: '#555' },
questionText: { fontSize: '18px', fontWeight: 'bold', marginBottom: '20px' },
optionsContainer: { display: 'flex', flexDirection: 'column', gap: '10px' },
optionButton: {
  padding: '10px',
  fontSize: '14px',
  backgroundColor: '#007bff',
  color: '#fff',
  border: 'none',
  borderRadius: '5px',
  cursor: 'pointer'
},
button: {
  padding: '10px 20px',
  backgroundColor: '#28a745',
  color: '#fff',
  border: 'none',
  borderRadius: '5px',
```

```
    cursor: 'pointer'
  }
};
```

**OUTPUT**

**1. Active Question Screen (Question State):**

Question 1 / 3

Which algorithm is commonly used for classification in ML?

[ Linear Regression ]
[ Support Vector Machines (SVM) ]
[ K-Means Clustering ]
[ PCA ]

**2. Quiz End Result Screen (Score State):**

**Plaintext:**

Quiz Complete!

You scored 3 out of 3

[ Restart Quiz ]